

Neural Networks 1

The Perceptron, the Delta Rule, and Where It Breaks

Where we are

- Previously: fitting a model with parameters, one input, one sigmoid-shaped curve
- Today: the simplest possible neural network — a single unit, called a **perceptron**
- Next week: the one thing today's network *can't* do, and how a second layer fixes it

-Now, a demo. I'm going to show you some images really quickly, and ask you about them. Ready?

How hard could computer vision be?

The assignment

“In 1966, Minsky hired a first-year undergraduate student and assigned him a problem to solve over the summer: connect a television camera to a computer and get the machine to describe what it sees.”

— Crevier, 1993, p. 88



Marvin Minsky
MIT

The summer project

MASSACHUSETTS INSTITUTE OF TECHNOLOGY
PROJECT MAC

Artificial Intelligence Group
Vision Memo. No. 100.

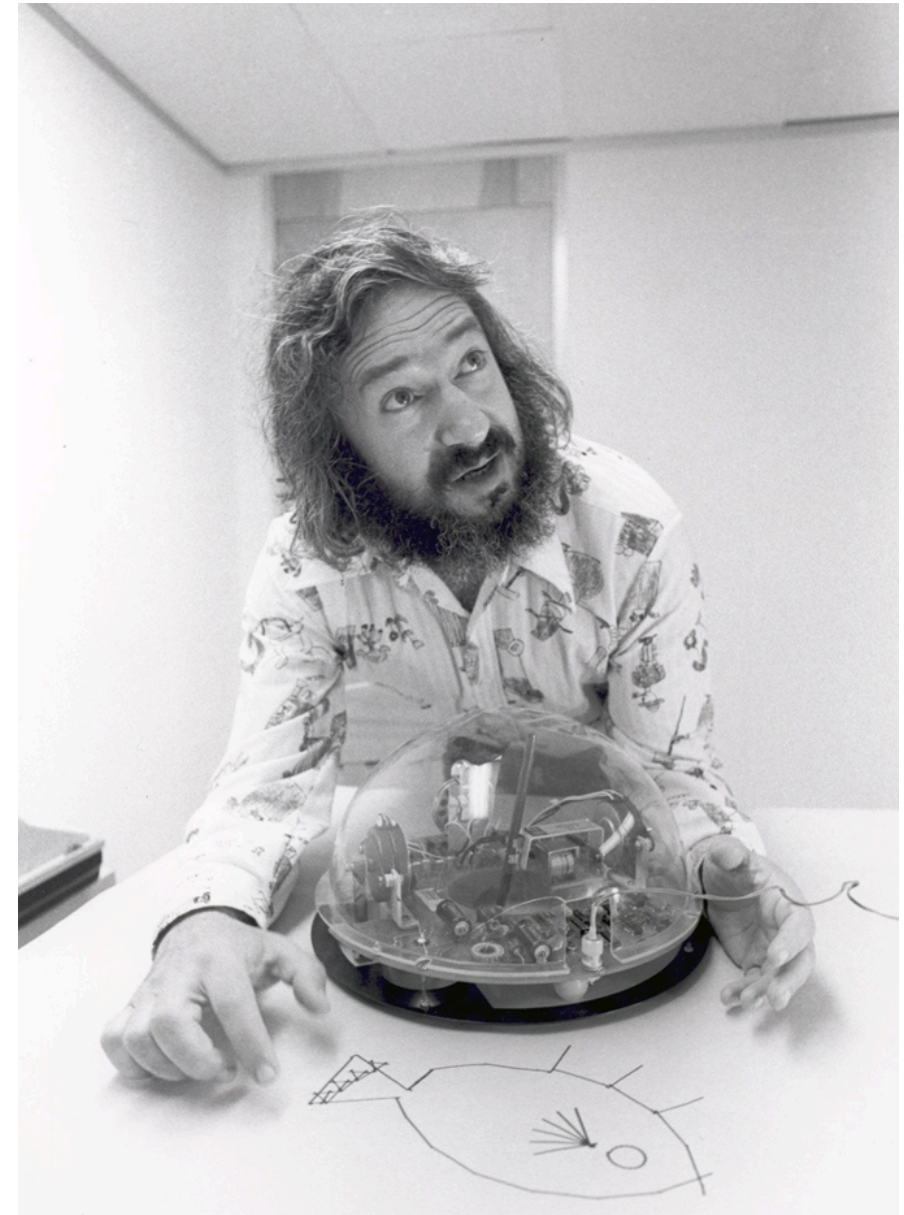
July 7, 1966

THE SUMMER VISION PROJECT

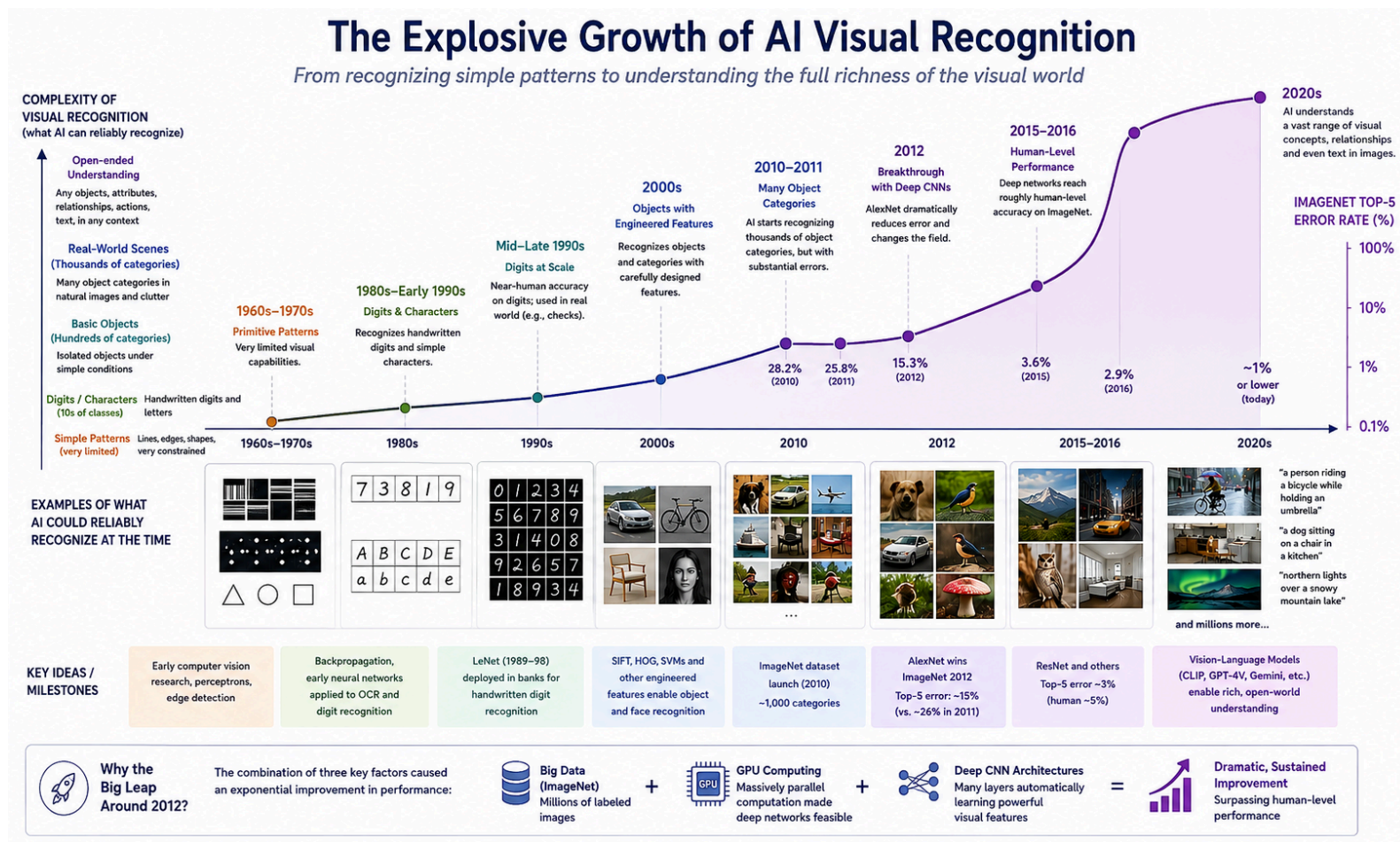
Seymour Papert

The summer vision project is an attempt to use our summer workers effectively in the construction of a significant part of a visual system. The particular task was chosen partly because it can be segmented into sub-problems which will allow individuals to work independently and yet participate in the construction of a system complex enough to be a real landmark in the development of "pattern recognition".

Seymour Papert



AI improvements since the 1960s



- Recognizing objects from images at human-level accuracy around 2012 with AlexNet, almost 50 years after the proposed 'summer project'
- image generated by ChatGPT

How hard could computer vision be?

The assignment

“In 1966, Minsky hired a first-year undergraduate student and assigned him a problem to solve over the summer: connect a television camera to a computer and get the machine to describe what it sees.”

— Crevier, 1993, p. 88

The problem

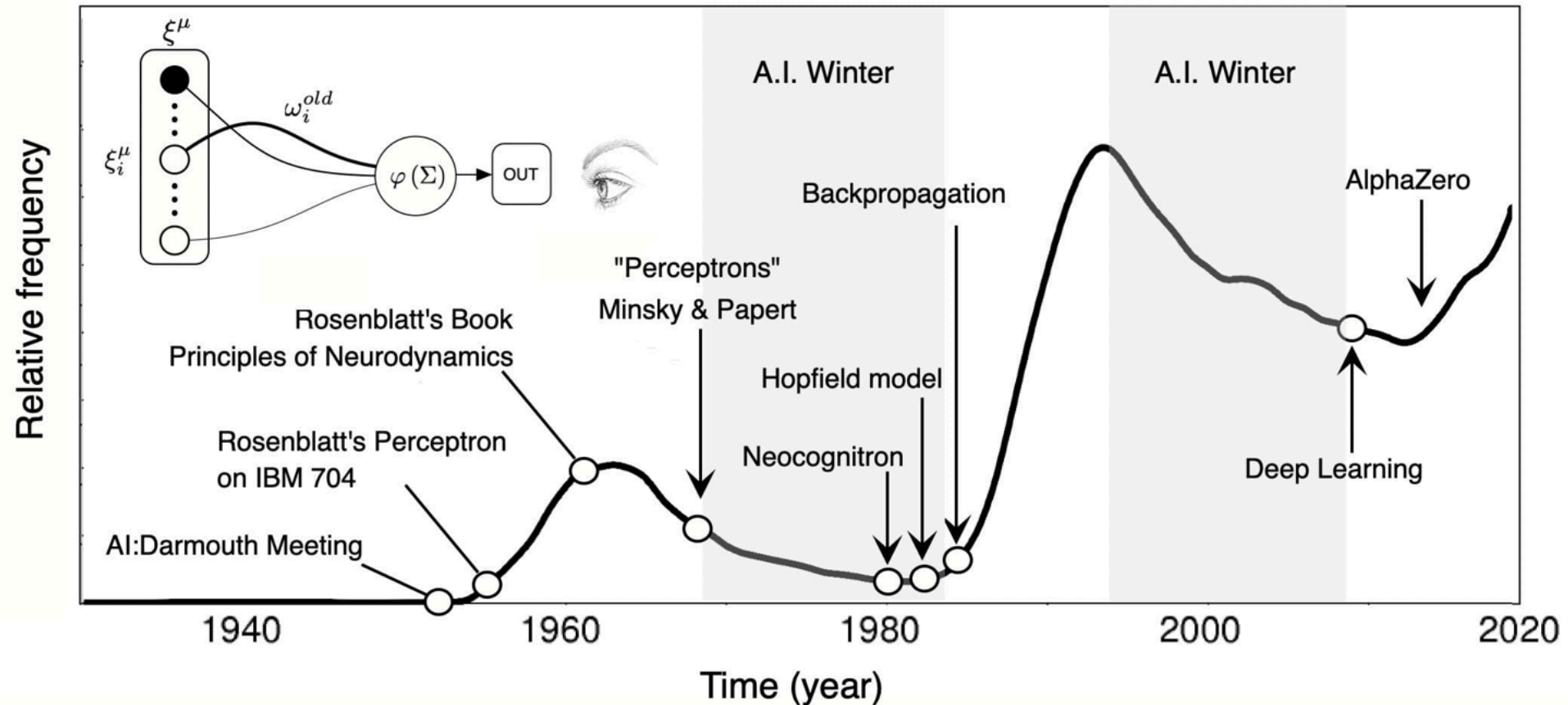
They didn't yet know what all the problems were.

Some of those problems only became apparent when they tried to solve them.



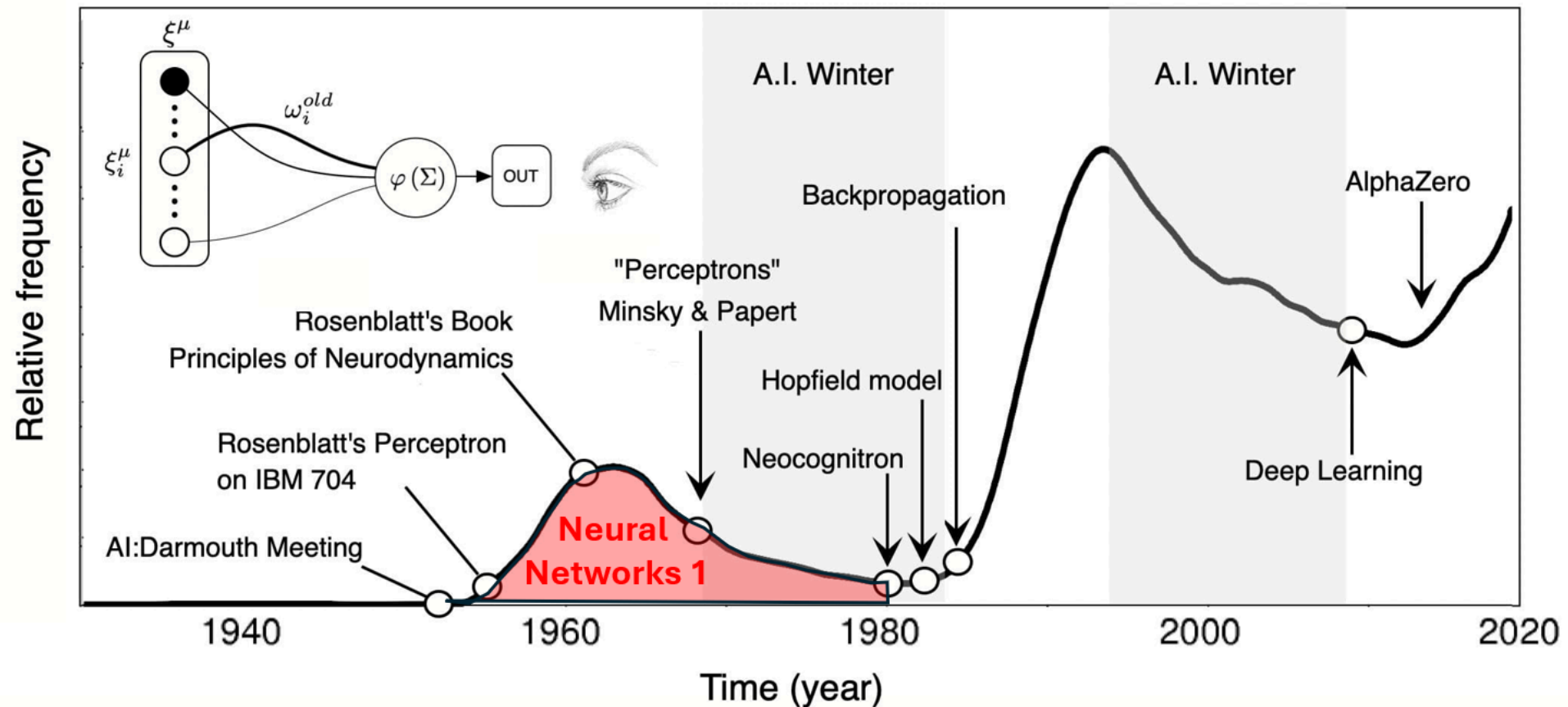
Marvin Minsky
MIT

The springs and winters of AI development



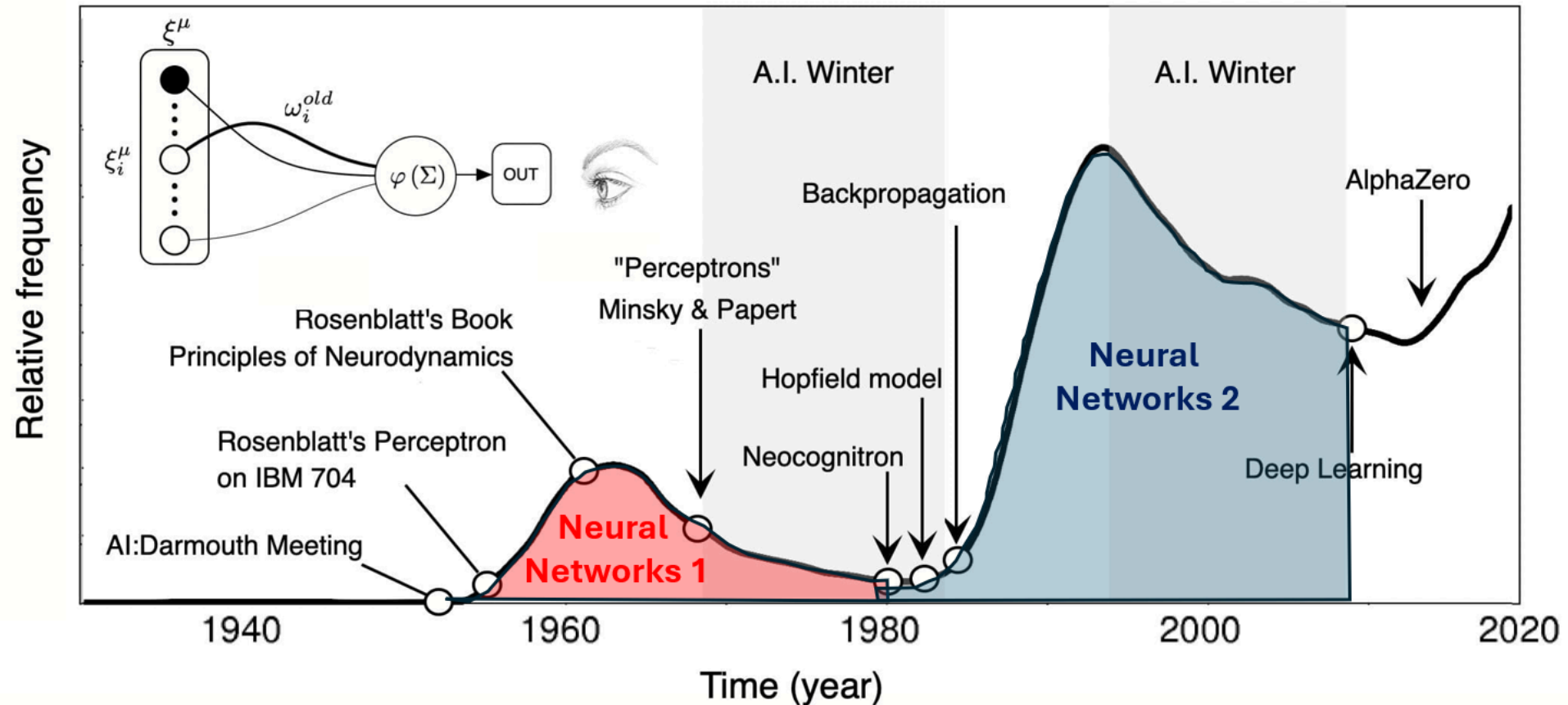
https://x.com/ricard_sole/status/1497306702181548039

The springs and winters of AI development



https://x.com/ricard_sole/status/1497306702181548039

The springs and winters of AI development



https://x.com/ricard_sole/status/1497306702181548039

Neural Networks 1, the bigger picture

- Everything today is intentionally small: 2 inputs, 1 unit, 16 data points — small enough to work through by hand if you wanted to
- The mechanism: nudge every weight a little after each example, in whatever direction reduces the error. This is genuinely the same mechanism behind large language models and most of modern deep learning
- What changes from here is scale, not kind: billions of weights instead of 3, many stacked layers instead of one unit, vastly more training data — but the core learning signal is the same “how wrong was I, and which way should each weight move?” idea you’ll build from scratch today
- So: today isn’t a simplified *toy version* of what real networks do. It’s the actual mechanism, just before it gets scaled up enormously

Learning objectives

By the end of today you should be able to:

1. Explain the core vocabulary of connectionist models: units, weights, activation
2. Build a single-unit perceptron and explain how it relates to the model built previously
3. Explain how the delta rule reduces error a little after each example, and how it relates to RMSD and `optim()` from previous weeks
4. Train a perceptron on a simple categorization task and watch its decision boundary change
5. Encounter the XOR problem (i.e., one of the causes of the first AI winter) firsthand, and predict whether today's kind of model can solve it

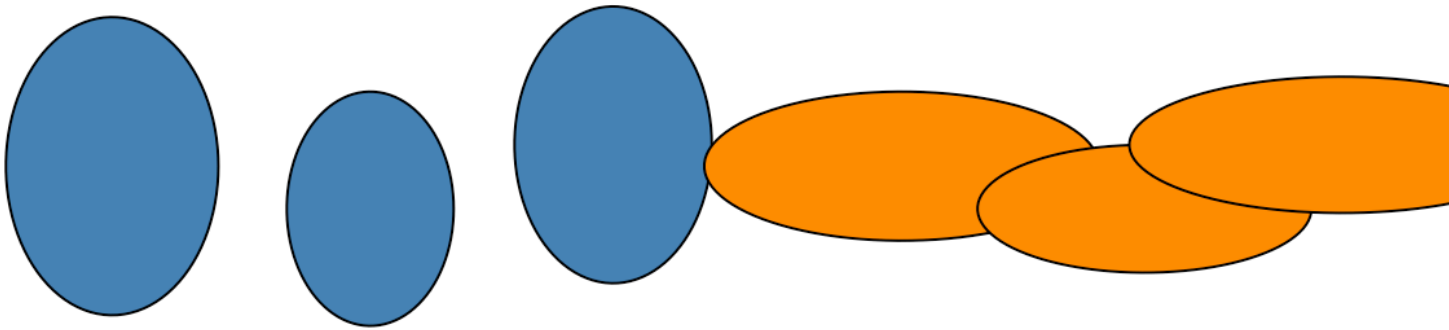
What is connectionism?

- A family of models that represent information as patterns of activation across simple **units**, connected by **weights**
- Loosely inspired by neurons and synapses; not a claim of biological realism, but a useful metaphor
- The core idea: **learning happens by changing the weights** to predict data (i.e., data-driven) rather than by rewriting rules (i.e., theory driven)
- This is a genuinely different way of building a model: instead of writing down one specific equation for the *whole* pattern of behavior in advance (like last week's Weibull curve), we use a generic equation that *learns* the pattern from training data

Example data set

Categorizing shapes

What do 'roundness' and 'elongation' look like?



High roundness, low elongation

Low roundness, high elongation

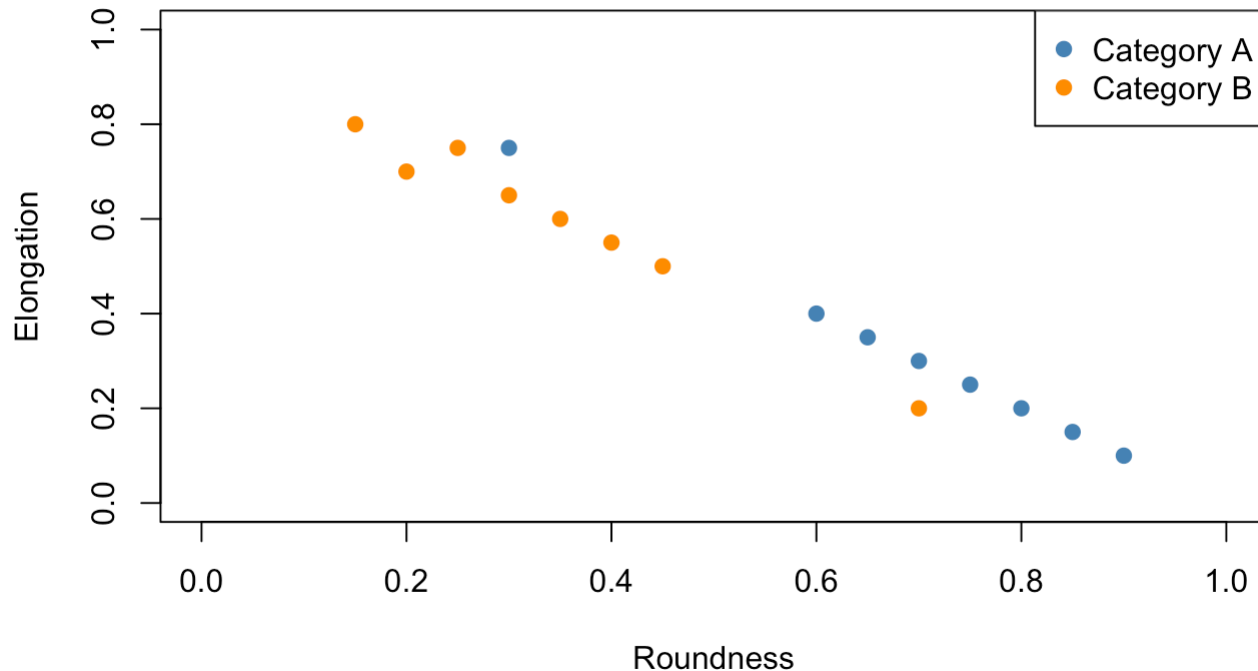
Categorizing shapes: the data

```
1 # 16 shapes, each described by two numbers (roundness, elongation) and a category
2 # label. First 8 rows are Category A (round, not elongated); last 8 are Category B
3 # (elongated, not round) -- with two rows deliberately mislabeled, same "add a
4 # little noise" convention as every dataset so far this course.
5 roundness <- c(0.9, 0.85, 0.8, 0.75, 0.7, 0.65, 0.6, 0.3,
6               0.2, 0.15, 0.25, 0.3, 0.35, 0.4, 0.45, 0.7)
7 elongation <- c(0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.75,
8               0.7, 0.8, 0.75, 0.65, 0.6, 0.55, 0.5, 0.2)
9 category <- c(1, 1, 1, 1, 1, 1, 1, 1,
10             0, 0, 0, 0, 0, 0, 0, 0)
11
12 data <- data.frame(roundness, elongation, category)
13 data
```

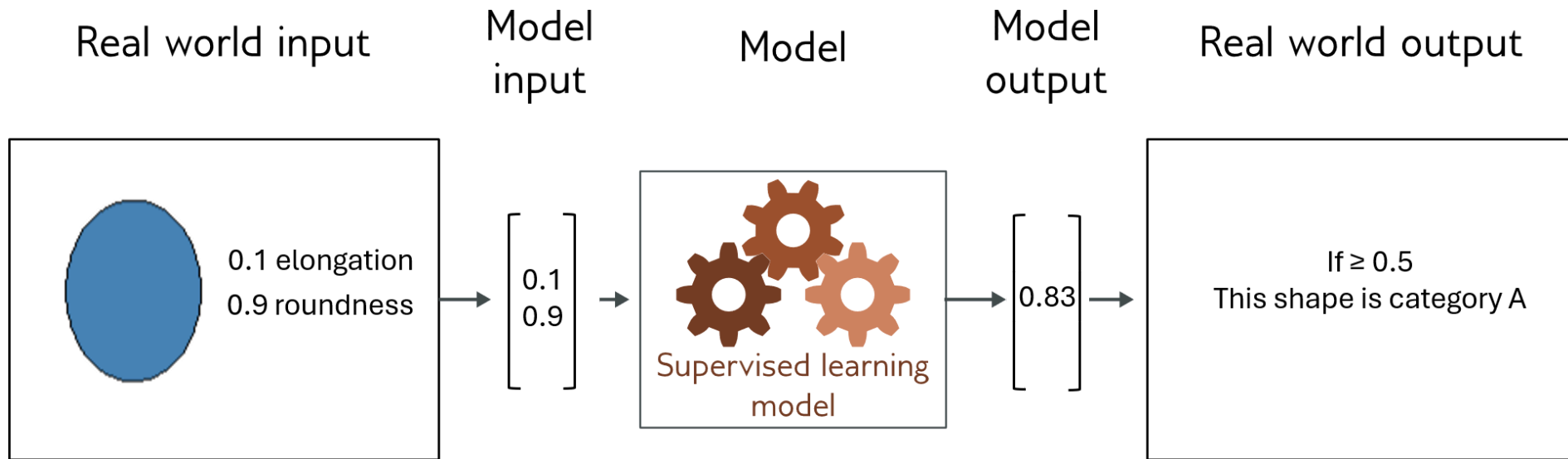
	roundness	elongation	category
1	0.90	0.10	1
2	0.85	0.15	1
3	0.80	0.20	1
4	0.75	0.25	1
5	0.70	0.30	1
6	0.65	0.35	1
7	0.60	0.40	1
8	0.30	0.75	1
9	0.20	0.70	0
10	0.15	0.80	0
11	0.25	0.75	0
12	0.30	0.65	0
13	0.35	0.60	0
14	0.40	0.55	0
15	0.45	0.50	0
16	0.70	0.20	0

Categorizing shapes: the data, scatter plot

```
1 # Scatter plot: one point per shape, colored by its true category. This is the
2 # same plotting pattern (points colored by category, legend in the corner) used
3 # for every dataset in this course.
4 plot(data$roundness, data$elongation, pch = 19,
5       col = ifelse(data$category == 1, "steelblue", "darkorange"),
6       xlab = "Roundness", ylab = "Elongation", xlim = c(0, 1), ylim = c(0, 1))
7 legend("topright", legend = c("Category A", "Category B"),
8       col = c("steelblue", "darkorange"), pch = 19)
```



From a shape to a category: the pipeline

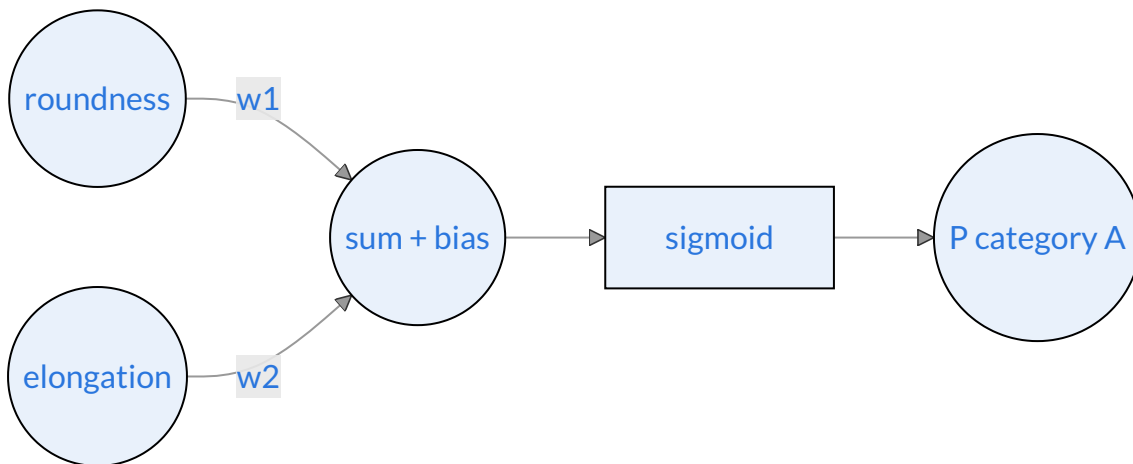


- Five boxes, same pipeline every time: a real thing in the world → turned into numbers → run through some model → a number comes out → turned back into a real-world statement
- That “number” in box four isn’t literally 1 or 0, it’s a probability like the 0.83 shown here. Turning a probability into an actual category is one more small step, which gets its own slide shortly
- Right now, box three is a mystery

**training a perceptron to
distinguish categories**

The single unit: a perceptron (1/2)

A **perceptron** is the simplest possible unit in a neural network: it takes one or more inputs, multiplies each by its own weight, adds a bias, sums everything together, and passes that sum through an activation function (here, a sigmoid) to produce an output.



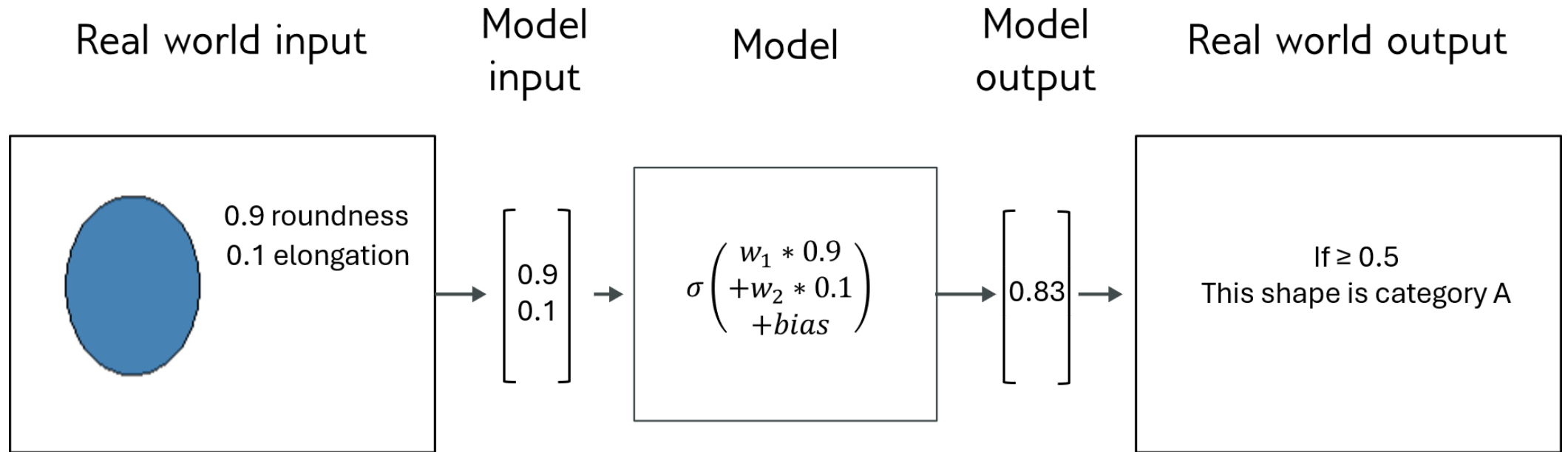
- Two inputs, each with its own weight; a bias term; sum them up; run the result through a sigmoid

The single unit: a perceptron (2/2)

- The specific choice of sigmoid isn't the important part. What matters is that it's **nonlinear**. Modern networks often use a different, simpler nonlinearity called **ReLU** (rectified linear unit): take the weighted sum plus bias, and if it's negative, output zero; if it's positive, output the value unchanged. Sigmoid and ReLU look very different, but both are nonlinear

```
1 # sigmoid(): squashes any real number into a probability between 0 and 1.
2 sigmoid <- function(x) {
3   1 / (1 + exp(-x))
4 }
5
6 # perceptron(): the full model. Combine both inputs (weighted) plus a bias into
7 # one number (this is the "which side of the boundary, how far" part), then
8 # squash that number into a probability with sigmoid (the "how confident" part).
9 perceptron <- function(w1, w2, bias, x1, x2) {
10   sigmoid(w1 * x1 + w2 * x2 + bias)
11 }
```

The pipeline, mystery solved



- Same five boxes as before. box three becomes the perceptron equation from the last slide
- pipeline: real thing $\rightarrow$ numbers $\rightarrow$ equation $\rightarrow$ number $\rightarrow$ real statement

From a number to a category: thresholding

- The perceptron's raw output is a **continuous probability** between 0 and 1 and not the category label itself
- Turning it into an actual decision takes one more step: **threshold at 0.5**

```
1 # Try one example shape (high roundness, low elongation) with a hand-picked set
2 # of weights, then apply the 0.5 threshold to turn the probability into a label.
3 example_output <- perceptron(w1 = 2, w2 = -2, bias = 0, x1 = 0.9, x2 = 0.1)
4 example_output
```

```
[1] 0.8320184
```

```
1 ifelse(example_output >= 0.5, "Category A", "Category B")
```

```
[1] "Category A"
```

From one input to two

$$p(x) = 1 - e^{-(x/\alpha)^\beta} \quad (\text{previously: one input, Weibull})$$

$$p(\text{category A}) = \frac{1}{1 + e^{-(w_1 \cdot \text{roundness} + w_2 \cdot \text{elongation} + \text{bias})}} \quad (\text{today: two inputs, sigmoid})$$

- Same overall idea: combine inputs into a single number, pass it through an S-shaped function, get a probability out
- The “parameters” are now called **weights** and a **bias**, which are numbers we don’t know yet and need to estimate from data
- Worth naming the task itself too: previously we did **regression**, i.e., fitting a continuous curve to a continuous proportion correct along a contrast scale. Today is **classification**, i.e., assigning each shape to one of two discrete categories. Both compute a continuous probability via a sigmoid; what differs is what we do with that number, which is to fit a smooth curve, or threshold it into a category label

How wrong is a guess?

```
1 # rmsd(): unpack a candidate (w1, w2, bias) triple from params, get every shape's
2 # predicted probability, then compare to the true 0/1 categories with RMSD --
3 # exactly the same discrepancy function used every week so far, just fed a
4 # perceptron's predictions instead of a curve's.
5 rmsd <- function(params, data) {
6   w1 <- params[1]; w2 <- params[2]; bias <- params[3]
7   predicted <- perceptron(w1, w2, bias, data$roundness, data$elongation)
8   sqrt(mean((data$category - predicted)^2))
9 }
10
11 rmsd(c(1, 1, 0), data)      # a so-so guess
```

```
[1] 0.5430748
```

```
1 rmsd(c(6, -6, 0), data)    # a much better guess
```

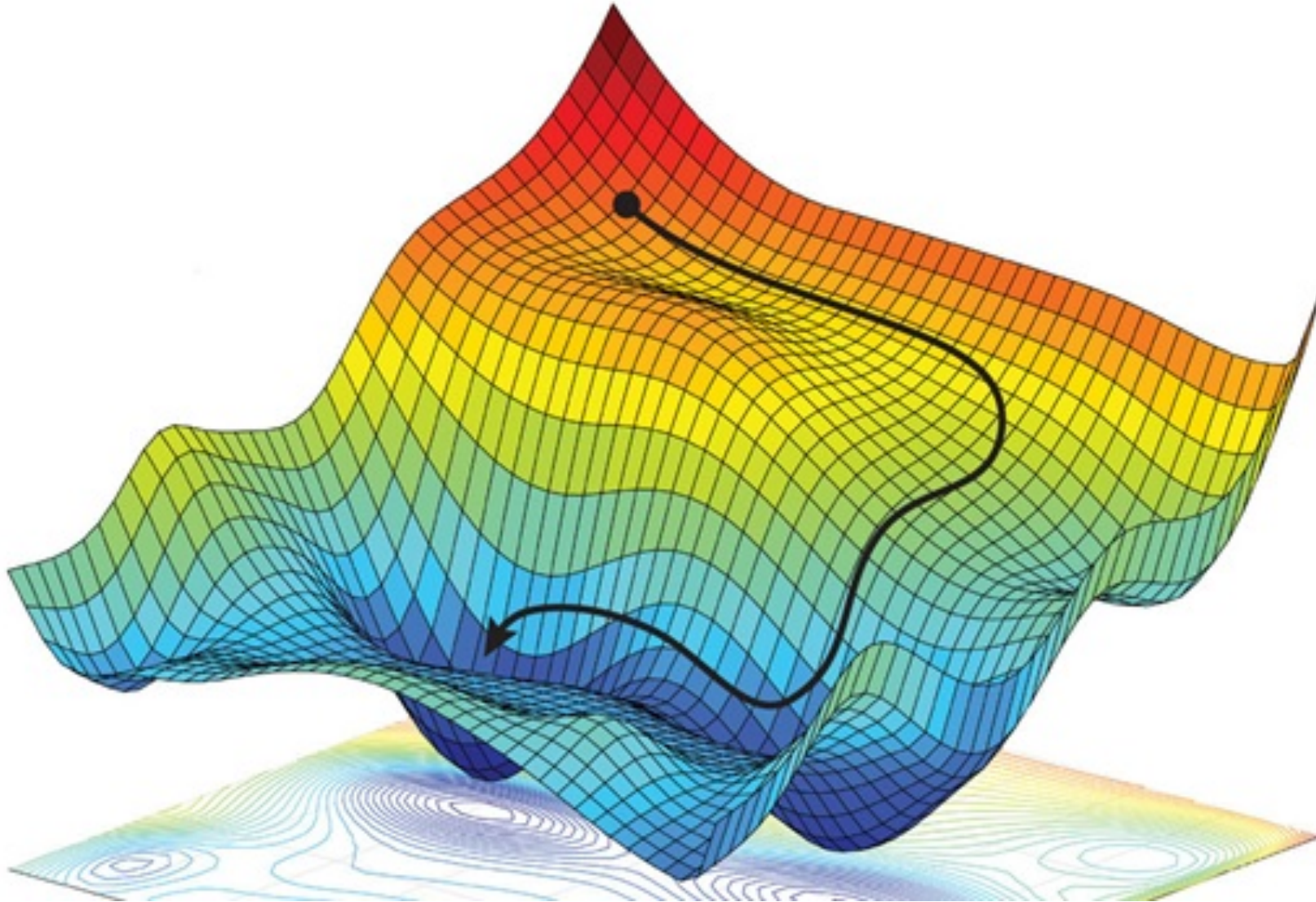
```
[1] 0.3694591
```

- Exactly the same RMSD used previously, applied to a different model
- We could absolutely hand this to `optim()`. You could try it yourself later — Today we'll learn it a different way, one that scales much better once networks get bigger

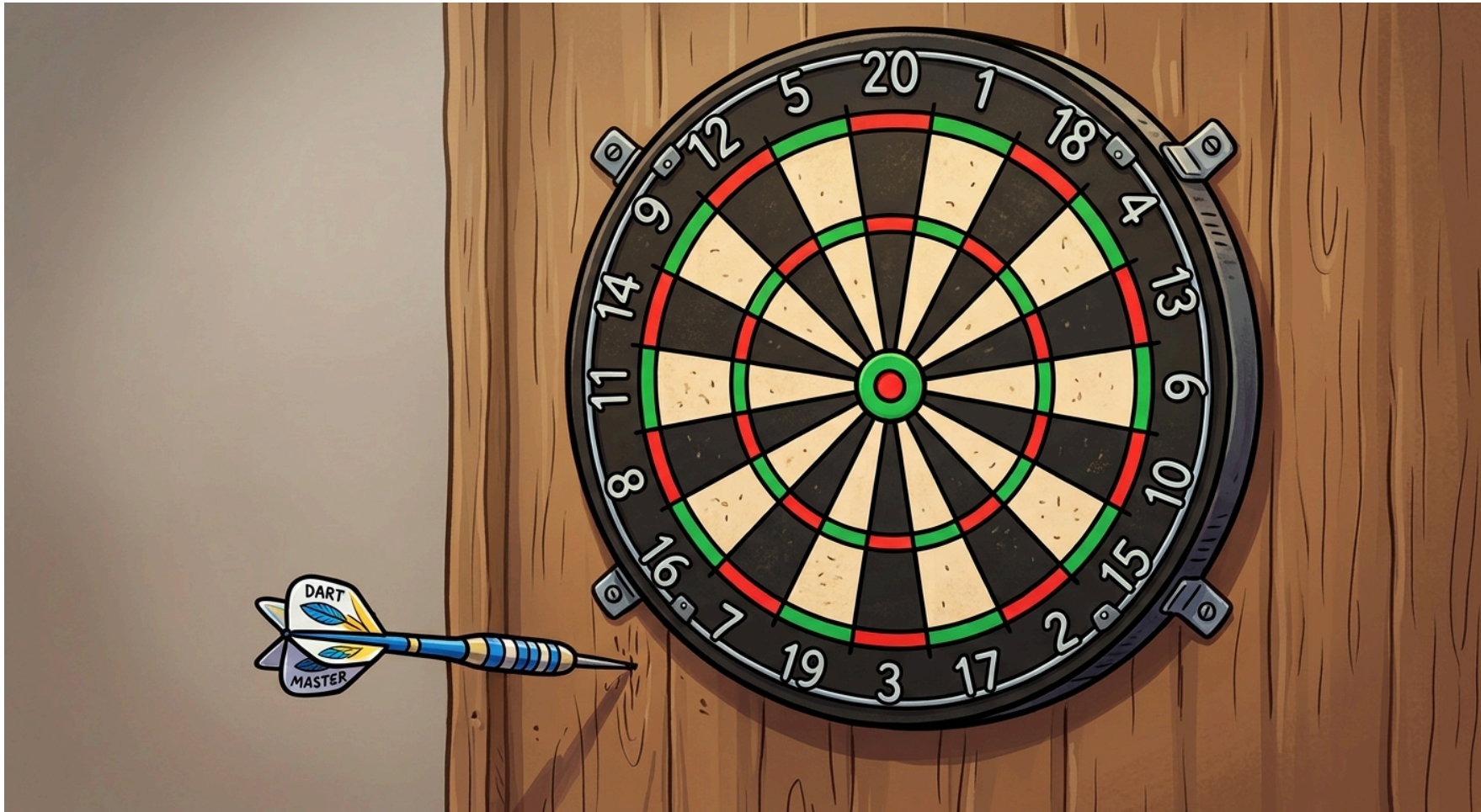
Learning from mistakes: adjusting the weights

- dart throwing story
- story applied to perceptron
- the delta rule
- applying the delta rule

Going down the error landscape



Before adjusting the neural networks weights: a dart-throwing story



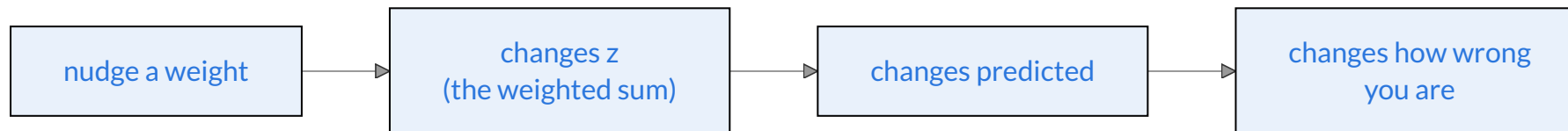
Before adjusting the neural networks weights: a dart-throwing story

- You throw a dart. It lands short and a bit low. Before your next throw, what do you actually need to know to adjust well?

A weight can't fix the error directly

$$p(\text{Category A}) = \sigma(w_1 \cdot \text{roundness} + w_2 \cdot \text{elongation} + b) = \sigma(z)$$

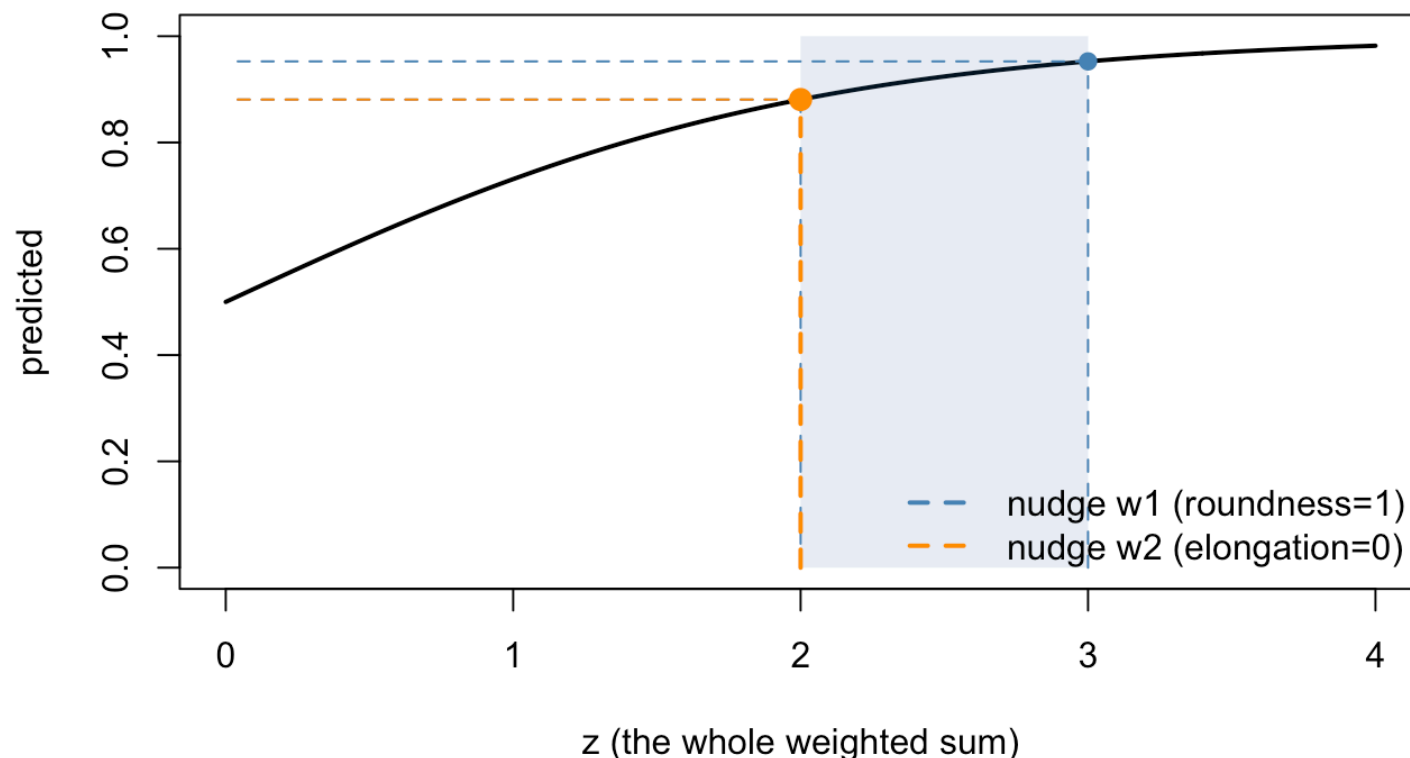
A weight doesn't affect how wrong you were all in one step. The weight has to get there through two doors:



- **Door 1** (between “nudge the weight” and “changes z”): how much does nudging *this* weight actually move **z**? This is the dart question “does it matter which part of my technique I fix”; some weights have a lot of leverage on **z** this trial, some have none at all
- **Door 2** (between “changes z” and “changes predicted”): given that **z** moved, how much did **predicted** actually move? This is “am i already confident in my answer, or am I still guessing?”
- Both doors have to be accounted for before you know how hard to push a given weight. That’s the whole content of the delta rule, coming up shortly

Watch door 1 in action (1/2)

One trial (roundness = 1, elongation = 0), nudging **one weight at a time** by the same amount, watching the *whole* $z = w1*roundness + w2*elongation + bias$ respond.



Watch door 1 in action (2/2)

- **Blue** (nudging w_1 , roundness's own weight from 2 to 3): z moves the full distance, $2 \rightarrow 3$; the shaded blue band shows exactly how much of the curve this nudge crosses, and **predicted** moves with it
- **Orange** (nudging w_2 , elongation's own weight, by the same amount, from 2 to 3): z doesn't move at all (it is still exactly 2) because elongation is 0 this trial, so w_2 's entire contribution to the sum is $w_2 \times 0$, always, no matter what w_2 is nudged to. There's no orange band, because there's no gap to shade
- side note: in this example, bias = 0, w_1 and w_2 are set to 2 and nudged to 3
- **This is the dart question “does it matter which part of my technique I fix?”** : a weight whose own input is present this trial has real leverage on the sum; a weight whose own input is zero has none at all, however hard you push it

Watch door 2 in action: The Numbers (1/3)

Same-sized nudge to z (+1.5 either way), dropped in from two different starting points:

```
1 sigmoid <- function(x) 1 / (1 + exp(-x))
2
3 cat(sprintf("Starting near z=0 (predicted near 0.5, uncertain):\n"))
```

```
1 cat(sprintf("  predicted at z=0:    %.4f\n", sigmoid(0)))
```

```
1 cat(sprintf("  predicted at z=1.5:  %.4f\n", sigmoid(1.5)))
```

```
1 cat(sprintf("  change in predicted: %.4f\n\n", sigmoid(1.5) - sigmoid(0)))
```

```
1 cat(sprintf("Starting near z=3 (predicted near 0.95, confident):\n"))
```

```
1 cat(sprintf("  predicted at z=3:    %.4f\n", sigmoid(3)))
```

```
1 cat(sprintf("  predicted at z=4.5:  %.4f\n", sigmoid(4.5)))
```

```
1 cat(sprintf("  change in predicted: %.4f\n", sigmoid(4.5) - sigmoid(3)))
```

Starting near $z=0$ (predicted near 0.5, uncertain):

predicted at $z=0$: 0.5000

predicted at $z=1.5$: 0.8176

change in predicted: 0.3176

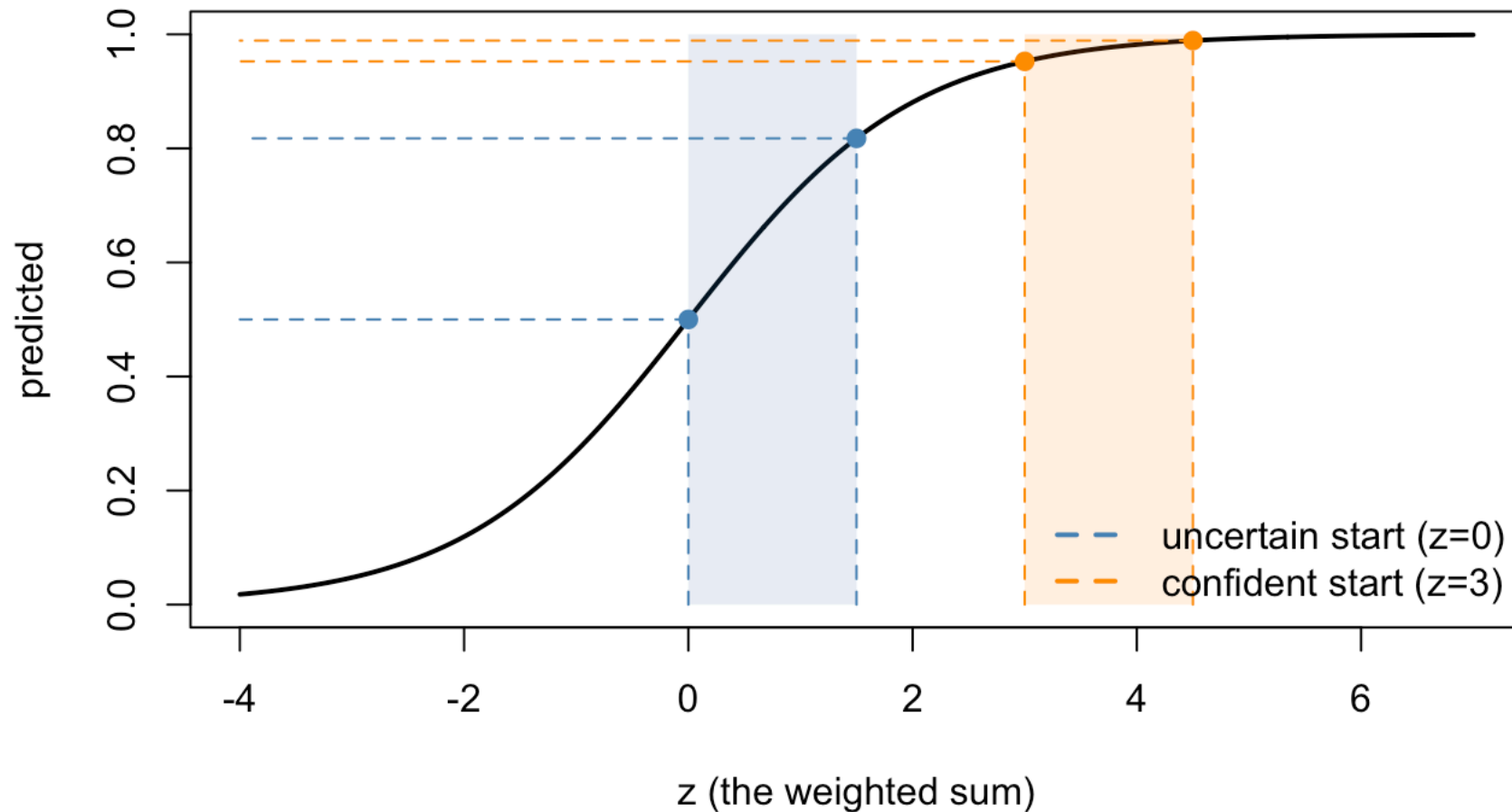
Starting near $z=3$ (predicted near 0.95, confident):

predicted at $z=3$: 0.9526

predicted at $z=4.5$: 0.9890

change in predicted: 0.0364

Watch door 2 in action: The plot (2/3)



Watch door 2 in action: The intuition

(3/3)

- The two shaded bands are the *same width* in *z* (both span 1.5) and that's the nudge, identical both times
- But look at the two blue dashed lines on the *predicted* axis versus the two orange dashed lines. The blue gap is much wider than the orange gap, even though the horizontal nudge was identical for both
- That vertical gap on the y-axis *is* door 2 (i.e, how far a nudge to *z* actually travels before it reaches *predicted*)

Learning as gradual error reduction: the delta rule (1/2)

- Previously we learned: `optim()` searches for the best weights all at once, considering the whole dataset every step. `optim()` worked fine in the Supervised Learning tutorial, fitting just two parameters (`alpha`, `beta`)
- But `optim()`'s default method (Nelder-Mead) doesn't scale well past a handful of parameters, and a real network can have thousands of weights. We need a method built to scale
- Today: the **delta rule** nudges the weights a little bit after *each individual training example*, in the direction that would have reduced the error on that one example, and it scales because it never needs to search blindly; it uses exactly the three dart questions from a few slides ago: how wrong was this guess, does it matter which part of my technique I fix (door 1), and am i already confident in my answer, or still guessing (door 2)
- (In the code you'll see a variable called `delta` in a few slides. It is worth knowing now that the delta in the code only holds two of these three questions; the third gets multiplied in separately. However, the delta rule combines all three questions. So the delta variable is part of the delta rule. More on why in a moment.)

Learning as gradual error reduction: the delta rule (2/2)

$$\Delta w = \eta \cdot (\text{target} - \text{predicted}) \cdot \text{predicted} \cdot (1 - \text{predicted}) \cdot \text{input}$$

- $(\text{target} - \text{predicted})$: the dart question: how wrong was this one guess, and in which direction?
- $\text{predicted} * (1 - \text{predicted})$: **door 2**, am i already confident in my answer, or still guessing?
- input : **door 1**, how much does nudging *this* weight actually move z ? If an input was 0 on this trial, its weight doesn't move at all this step, no matter how wrong the prediction was
- η (η), the **learning rate**: once both doors are accounted for, how hard do you actually push? Too large and training overshoots and bounces around; too small and training crawls.

A worked example: which weights actually move? (1/2)

Take one trial: roundness = 1, elongation = 0, target = category A (1). Current weights: $w_1 = 2$, $w_2 = -2$, $\text{bias} = 0$, learning rate $\eta = 0.5$. Watch each weight's nudge go through both doors.

```
1 w1 <- 2; w2 <- -2; bias <- 0 # current weights and bias
2 roundness_i <- 1; elongation_i <- 0; target <- 1 # this trial's inputs and true category
3 lr <- 0.5
4
5 predicted <- perceptron(w1, w2, bias, roundness_i, elongation_i) # forward pass: current guess
6 delta <- (target - predicted) * predicted * (1 - predicted) # error x sigmoid steepness
7
8 w1_new <- w1 + lr * delta * roundness_i # scaled by roundness (this weight's own input)
9 w2_new <- w2 + lr * delta * elongation_i # scaled by elongation (zero on this trial)
10 bias_new <- bias + lr * delta # no input to scale by -- always updates
11
12 round(c(predicted = predicted, delta = delta,
13          w1 = w1_new, w2 = w2_new, bias = bias_new), 4)
```

predicted	delta	w1	w2	bias
0.8808	0.0125	2.0063	-2.0000	0.0063

A worked example: which weights actually move? (2/2)

- **w1** moves, because its own input (roundness = 1) is nonzero. **door 1 is wide open** for **w1** this trial: it gets the full correction: **w1** goes from 2 to 2.0063
- **w2** doesn't move **at all**. **w2's** own input (elongation = 0) means **door 1 is shut** for **w2** this trial, zeroing out its entire update, so **w2** stays exactly at -2
- **bias** moves too, from 0 to 0.0063, by coincidence *exactly* the same amount as **w1** moved. That's not a general rule: **bias** isn't multiplied by any input at all (equivalently, its "input" is always 1, door 1 is always fully open for bias), and roundness happened to also equal exactly 1 on this trial. If roundness had been 0.5 instead, **w1** would have moved only half as far as **bias** did
- Notice **delta** itself (0.0125) is the *same* number for every weight in this update. That's door 2, already multiplied in before door 1 ever gets applied: **delta** already includes **predicted * (1 - predicted)**, the sigmoid-steepness factor from a few slides ago. Door 1 (the **roundness_i / elongation_i** multiplication below) is what happens *after* that, separately for each weight

Did that one step actually help?

The whole point of the update was to make the prediction less wrong. Let's check, using the exact same trial:

```
1 # predicted_new: run the SAME trial back through the perceptron, but with the
2 # just-updated weights, to see what the model would say now.
3 predicted_new <- perceptron(w1_new, w2_new, bias_new, roundness_i, elongation_i)
4
5 round(c(predicted = predicted, predicted_new = predicted_new, target = target), 4)
```

predicted	predicted_new	target
0.8808	0.8821	1.0000

```
1 round(c(error_before = abs(target - predicted),
2         error_after  = abs(target - predicted_new)), 4)
```

error_before	error_after
0.1192	0.1179

What if the unit starts out unsure? (1/2)

Let's take the same trial with the same inputs. But now imagine the weights happened to start somewhere that makes the unit genuinely uncertain (**predicted** near 0.5), instead of already fairly confident. This is door 2 again.

```
1 w1_b <- 0; w2_b <- -2; bias_b <- 0 # chosen so that z = 0 on this trial -> predicted = 0.5 exactly
2
3 predicted_b <- perceptron(w1_b, w2_b, bias_b, roundness_i, elongation_i)
4 delta_b <- (target - predicted_b) * predicted_b * (1 - predicted_b)
5
6 round(c(predicted = predicted_b, delta = delta_b), 4)
```

predicted	delta
0.500	0.125

```
1 round(c(delta_when_unsure = delta_b, delta_from_before = delta), 4)
```

delta_when_unsure	delta_from_before
0.1250	0.0125

What if the unit starts out unsure? (2/2)

- `predicted` here is exactly 0.5; the model is a coin flip, maximally uncertain
- `delta` comes out to 0.125; almost **10 times larger** than the 0.0125 we got from the original, already-fairly-confident starting point
- Same input, same target, same learning rate; the only thing that changed was how uncertain the unit was, and that alone produced a 10x difference in how hard the correction pushed
- Door 2, wide open this time, instead of nearly shut; not as an abstract formula, but as a real difference in how much a weight actually moves

What if we change the learning rate? (1/2)

Both doors are already accounted for in `delta`. `eta` is the separate question of how hard to actually push, once you know which direction and how much leverage you have. Back to the original starting weights (`w1 = 2`, `w2 = -2`, `bias = 0`). Same trial, same `delta` but now only `eta` changes.

```
1 for (test_lr in c(0.05, 0.5, 5)) {  
2   w1_new_lr <- w1 + test_lr * delta * roundness_i  
3   bias_new_lr <- bias + test_lr * delta  
4   predicted_new_lr <- perceptron(w1_new_lr, w2, bias_new_lr, roundness_i, elongation_i)  
5   cat(sprintf("eta = %-5s ->  w1: %.4f   predicted_new: %.4f\n",  
6               test_lr, w1_new_lr, predicted_new_lr))  
7 }
```

```
eta = 0.05  ->  w1: 2.0006   predicted_new: 0.8809  
eta = 0.5   ->  w1: 2.0063   predicted_new: 0.8821  
eta = 5     ->  w1: 2.0626   predicted_new: 0.8933
```

What if we change the learning rate?

(2/2)

- `eta = 0.05`: barely moves, as `w1` goes from 2 to 2.0006, `predicted_new` is nearly identical to before
- `eta = 0.5`: the size we've been using is a small, steady step
- `eta = 5`: a much bigger jump in `w1`, and a correspondingly bigger jump in `predicted_new`
- In our example, `delta` is the same every time, and `eta` allows us controls how far to walk in the direction `delta` points. Too small and training crawls; too large and it can overshoot and bounce around instead of settling

Why `predicted * (1 - predicted)`?

You've now seen door 2 happen twice: once abstractly (the sigmoid curve, two colors), once concretely (the worked example, `w2` frozen because its door was shut). Door 2 turns out to be the derivative (or slope) of the sigmoid (or more generally, the nonlinearity) at that point:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \Rightarrow \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- `predicted` is $\sigma(x)$ at the current weights, so `predicted * (1 - predicted)` is $\sigma'(x)$. Not an approximation of the slope, not a coincidence, it's the actual slope of the sigmoid at that point
- This is exactly why door 2's gap was bigger near the middle and smaller near the shoulders in both figures: the sigmoid is genuinely steeper there, and this term *is* that steepness, not a plausible-sounding stand-in for it

**Is this actually gradient
descent?**

Is this actually gradient descent?

Nelder-Mead (`optim()`)

- **Not gradient descent:** Default method in `optim()`, but operates as a direct search.
- **Geometric trial-and-error:** Never computes a derivative: it compares RMSD at candidate points and shifts a simplex shape.
- **Metaphorical:** “Walk downhill” is only a loose heuristic metaphor here.
- **Dimensional Scaling** Collapses in high dimensions ($> 5 - 10$ parameters).

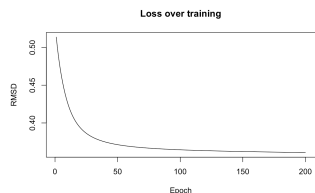
The Delta Rule

- **Calculus-driven:** Uses explicit derivatives for weight adjustments.
- **Exact slope:** $\text{predicted} * (1 - \text{predicted})$ is the exact calculus derivative of the sigmoid, giving direction and steepness.
- **Literal gradient descent:** “Walk downhill” is now mathematically true as we compute the exact slope.
- **Dimensional Scaling** Scales to many parameters, millions with back propagation (see Hinton (1992), and *Neural Networks 2*).

Watching the network learn

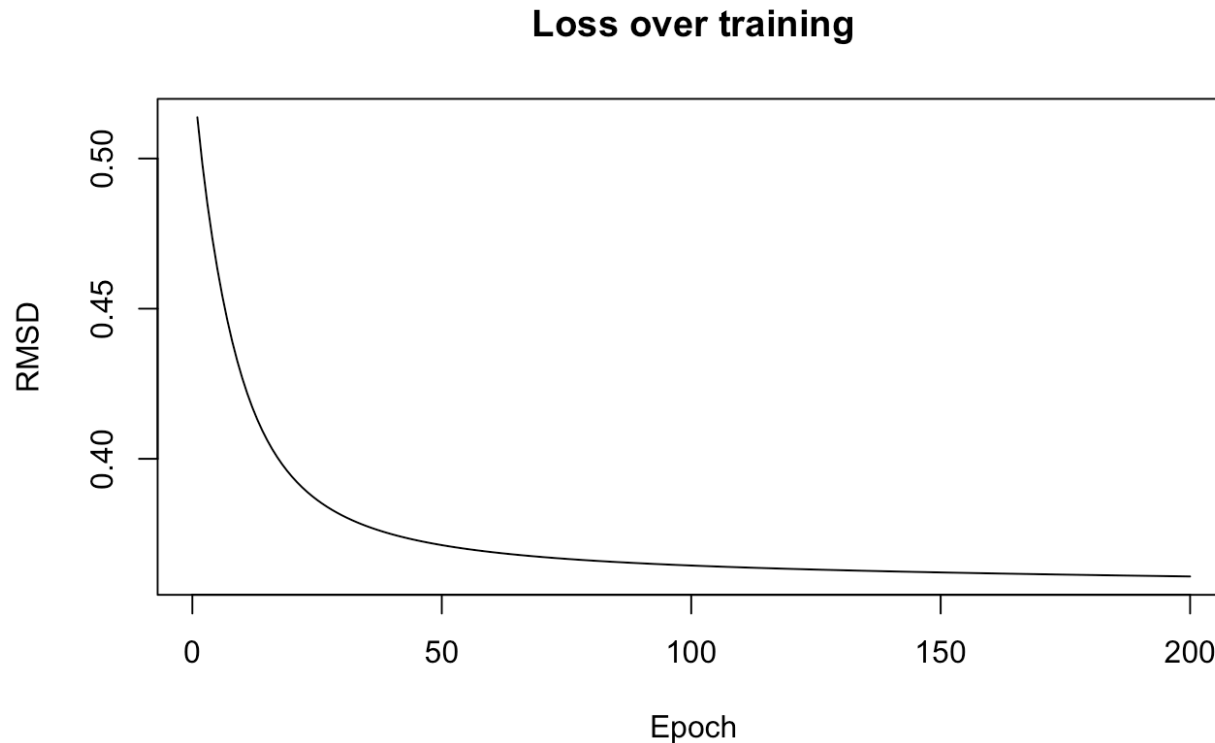
Watch it train: the loss curve (1/2)

```
1 set.seed(1)
2 # Start from small random weights (not zero -- a symmetric starting point can
3 # cause its own problems, same idea as previously).
4 w1 <- rnorm(1, sd = 0.5); w2 <- rnorm(1, sd = 0.5); bias <- rnorm(1, sd = 0.5)
5 lr <- 0.5
6 n_epochs <- 200
7 loss_history <- numeric(n_epochs) # one RMSD value per epoch, for the loss curve
8
9 for (epoch in 1:n_epochs) {
10   for (i in 1:nrow(data)) { # one trial = one row of the data, in order
11     xi1 <- data$roundness[i]; xi2 <- data$elongation[i]; target <- data$category[i]
12     pred <- perceptron(w1, w2, bias, xi1, xi2) # forward pass
13     delta <- (target - pred) * pred * (1 - pred) # error x sigmoid steepness
14     w1 <- w1 + lr * delta * xi1 # each weight scaled by its own input
15     w2 <- w2 + lr * delta * xi2
16     bias <- bias + lr * delta # bias has no input to scale by
17   }
18   # after all 16 shapes this epoch, compute RMSD across the whole dataset
19   loss_history[epoch] <- rmsd(c(w1, w2, bias), data)
20 }
21
22 plot(loss_history, type = "l", xlab = "Epoch", ylab = "RMSD", main = "Loss over training")
```



Watch it train: the loss curve (2/2)

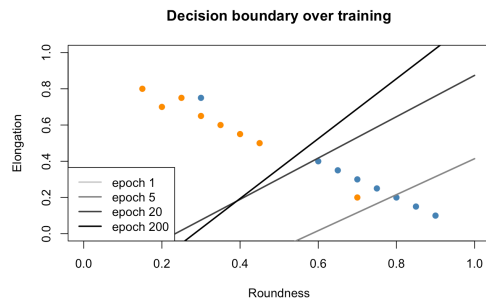
```
1 plot(loss_history, type = "l", xlab = "Epoch", ylab = "RMSD", main = "Loss over training")
```



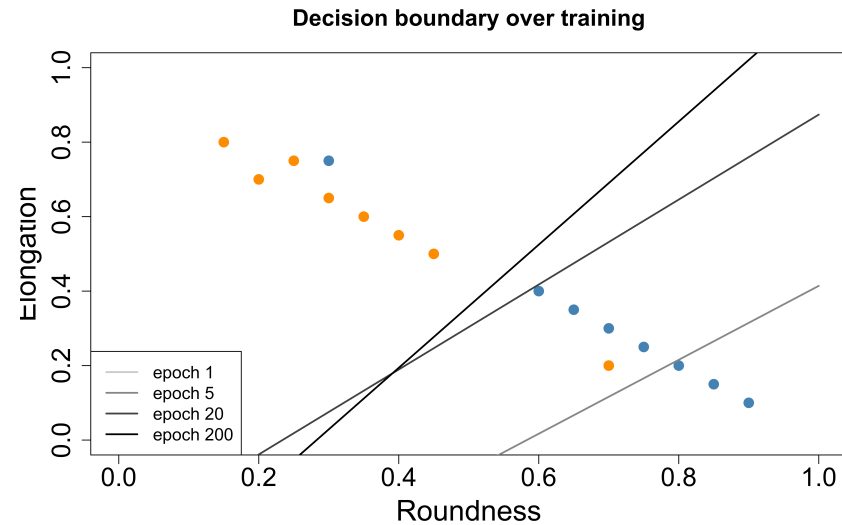
- One **epoch** = one full pass through every training example
- The loss should drop quickly at first, then level off

Watch it train: the decision boundary (1/2)

```
1 # Same training loop as above, but this time save the weights at a few specific
2 # epochs along the way, so we can draw the decision boundary at each checkpoint
3 # and watch it move.
4 checkpoints <- c(1, 5, 20, 200)
5 saved_weights <- list()
6
7 set.seed(1) # same seed as the loss-curve run, so this reproduces the same trajectory
8 w1 <- rnorm(1, sd = 0.5); w2 <- rnorm(1, sd = 0.5); bias <- rnorm(1, sd = 0.5)
9
10 for (epoch in 1:n_epochs) {
11   for (i in 1:nrow(data)) {
12     x11 <- data$roundness[i]; x12 <- data$elongation[i]; target <- data$category[i]
13     pred <- perceptron(w1, w2, bias, x11, x12)
14     delta <- (target - pred) * pred * (1 - pred)
15     w1 <- w1 + lr * delta * x11
16     w2 <- w2 + lr * delta * x12
17     bias <- bias + lr * delta
18   }
19   if (epoch %in% checkpoints) {
20     saved_weights[[as.character(epoch)]] <- c(w1, w2, bias) # snapshot this epoch's weights
21   }
22 }
23
24 plot(data$roundness, data$elongation, pch = 19,
25      col = ifelse(data$category == 1, "steelblue", "darkorange"),
26      xlab = "Roundness", ylab = "Elongation", xlim = c(0, 1), ylim = c(0, 1),
27      main = "Decision boundary over training")
28
29 boundary_colors <- colorRampPalette(c("gray80", "black"))(length(checkpoints))
30 x_seq <- seq(0, 1, length.out = 100)
31 for (k in seq_along(checkpoints)) {
32   wts <- saved_weights[[as.character(checkpoints[k])]]
33   # solve w1*x1 + w2*x2 + bias = 0 for x2 (elongation), given a range of x1 (roundness)
34   y_line <- -(wts[1] * x_seq + wts[3]) / wts[2]
35   lines(x_seq, y_line, col = boundary_colors[k], lwd = 2)
36 }
37 legend("bottomleft", legend = paste("epoch", checkpoints),
38      col = boundary_colors, lty = 1, lwd = 2)
```



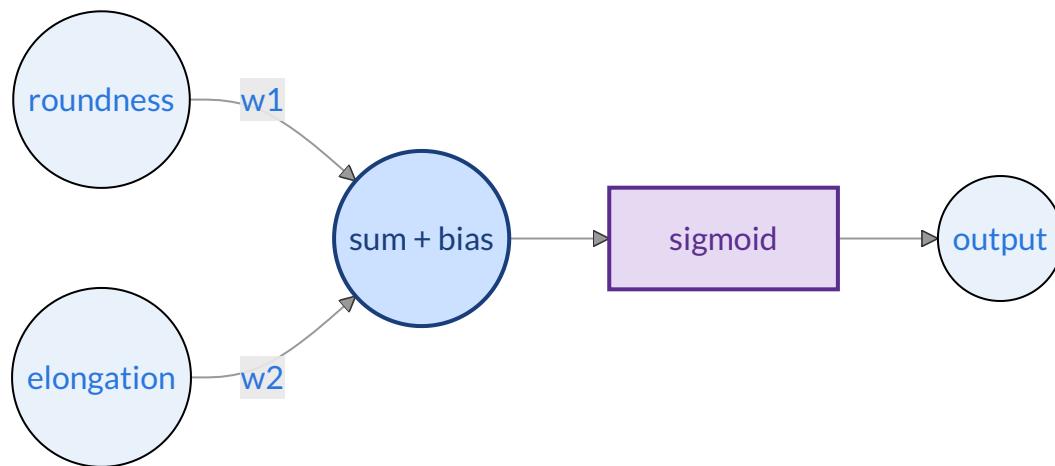
Watch it train: the decision boundary (2/2)



The trained perceptron: two parts, two jobs

Now that you've watched one train, here's what the two halves of the formula are actually doing:

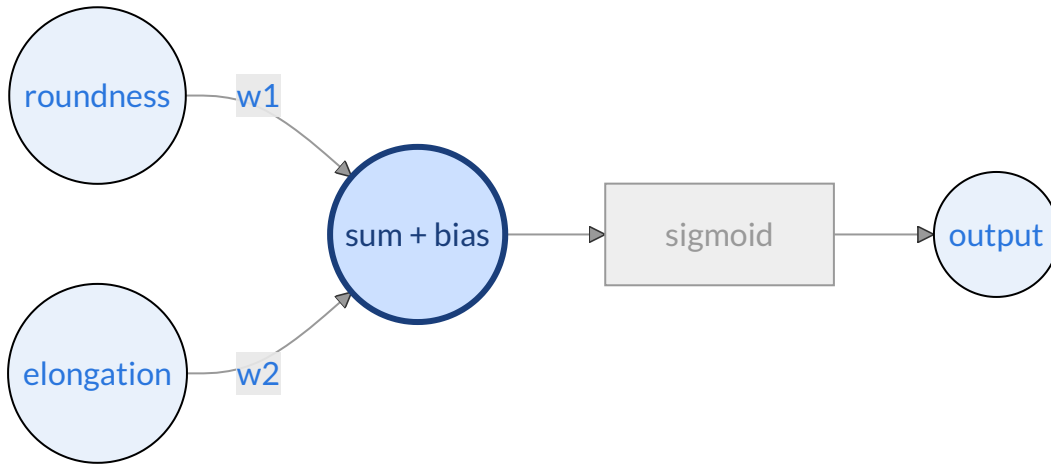
$$\text{output} = \sigma(w_1 x_1 + w_2 x_2 + b)$$



- The part in **navy** ($w_1 x_1 + w_2 x_2 + b$) is exactly the **decision boundary** you just watched rotate into place on the previous slide
- The part in **purple**, the sigmoid, σ , is the **nonlinearity**
- Two different jobs. Next two slides: one for each

The decision boundary

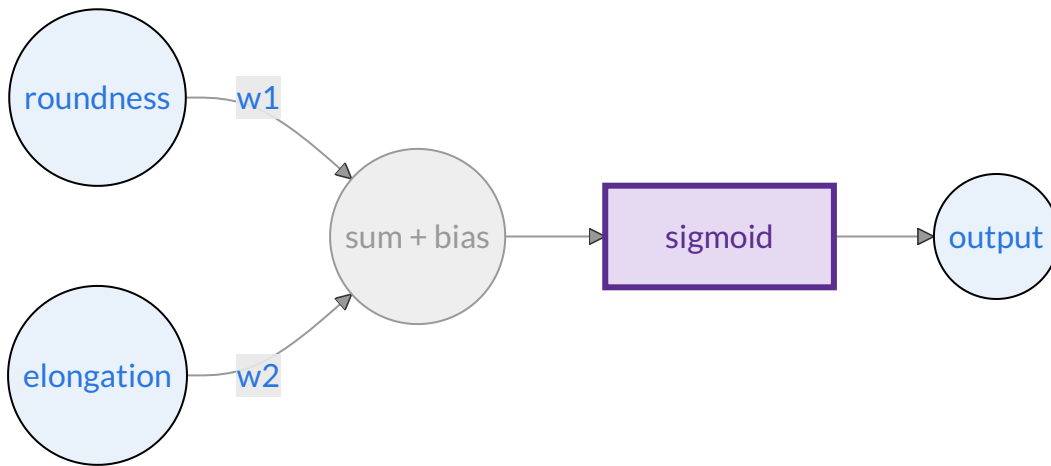
$$\text{output} = \sigma(w_1 x_1 + w_2 x_2 + b)$$



- $w_1 x_1 + w_2 x_2 + b$ collapses two inputs into a single number: positive on one side of the boundary, negative on the other, exactly zero right on it
- This part is purely geometric. Not a probability yet, just “which side, and how far”
- This is *exactly* the line you watched rotate into place on the previous slide, epoch by epoch.

The nonlinearity (1/2)

$$\text{output} = \sigma(w_1 x_1 + w_2 x_2 + b)$$



The sigmoid's job is different from the boundary's: it turns *distance from the boundary* into a **confidence level**.

The nonlinearity (1/2)

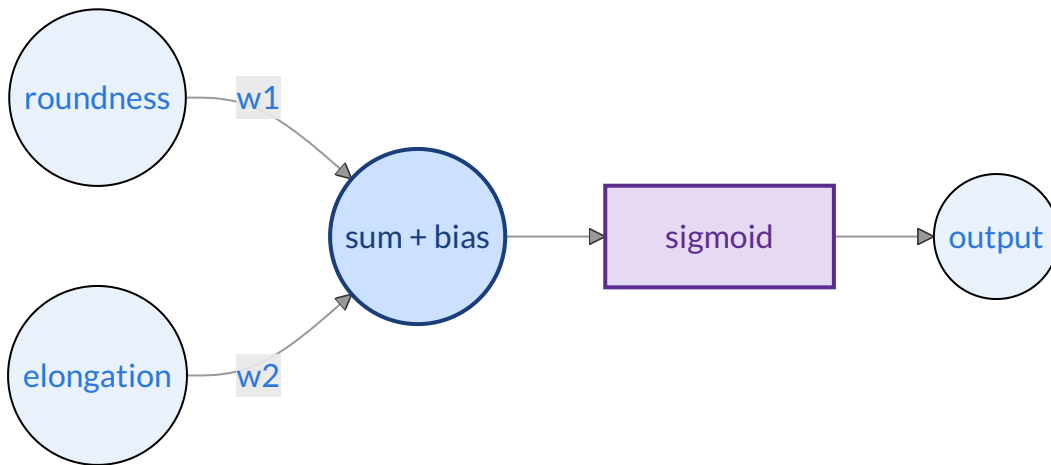
```
1 # Five points at varying distances from the trained decision boundary. z is the
2 # raw weighted sum (the "boundary" part, previous slide); predicted is what the
3 # sigmoid turns that distance into (the "confidence" part, this slide).
4 example_points <- data.frame(roundness = c(0.9, 0.6, 0.5, 0.4, 0.1),
5                               elongation = c(0.1, 0.4, 0.5, 0.6, 0.9))
6 example_points$z <- with(example_points, 2 * roundness - 2 * elongation)
7 example_points$predicted <- perceptron(2, -2, 0, example_points$roundness, example_points$elongation)
8 example_points
```

	roundness	elongation	z	predicted
1	0.9	0.1	1.6	0.8320184
2	0.6	0.4	0.4	0.5986877
3	0.5	0.5	0.0	0.5000000
4	0.4	0.6	-0.4	0.4013123
5	0.1	0.9	-1.6	0.1679816

- Far on the “roundness-heavy” side of the boundary ($z = 1.6$), the prediction is confidently high (0.83)
- Right on the boundary ($z = 0$), the prediction is exactly 0.5, which is maximum uncertainty, genuinely a coin flip. Here the input points fall directly on the decision boundary line.
- Far on the “elongation-heavy” side ($z = -1.6$), the prediction is confidently low (0.17)
- The pattern is symmetric around the boundary: a certain distance on one side gives a certain confidence toward category A; the same distance on the other side gives the *same* confidence, toward category B

Summary: The trained perceptron: two parts, two jobs

$$\text{output} = \sigma(w_1 x_1 + w_2 x_2 + b)$$



So: the **navy** part answers “which side of the boundary is this point on, and how far?”; the **purple** part translates “how far” into “how sure are we?” That’s also exactly why the correction term $\text{predicted} * (1 - \text{predicted})$ from a few slides from now peaks for points near the boundary and vanishes for points far from it. The delta rule automatically focuses its biggest updates on the cases the model is currently least sure about.

The limits of a single perceptron

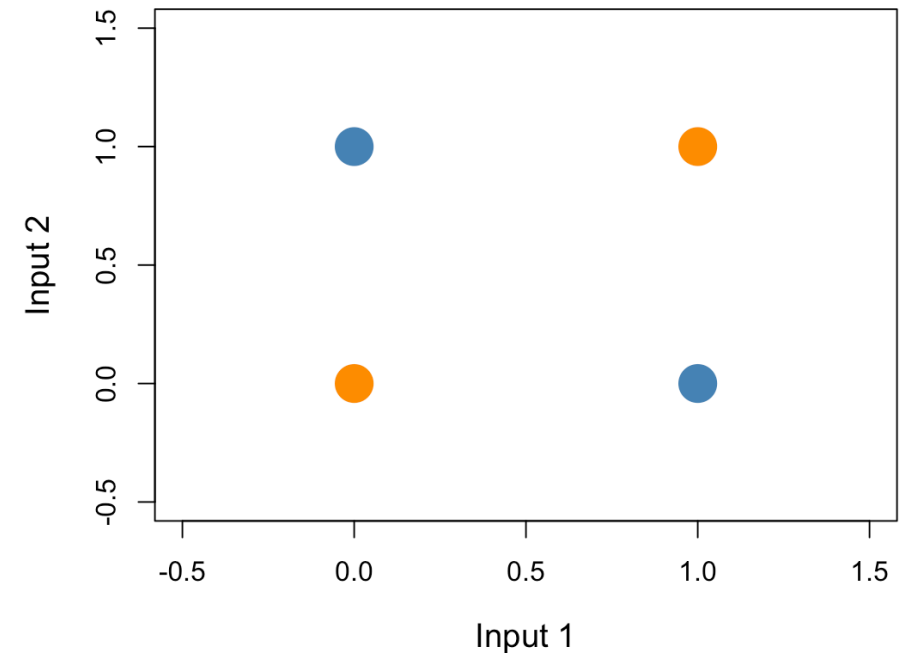
A puzzle: can a perceptron learn this?

Consider a much smaller problem with just two inputs, each either 0 or 1:

XOR Data

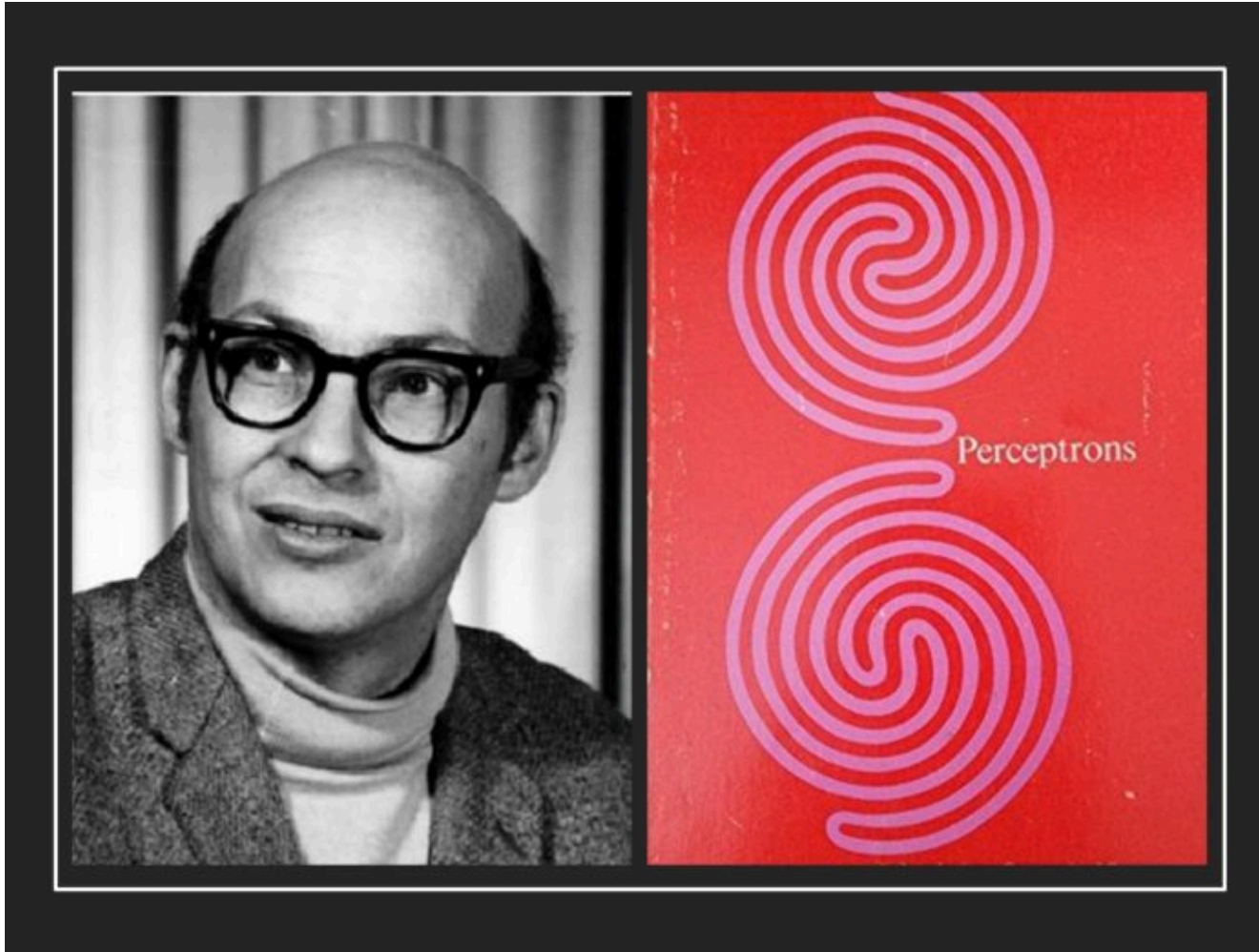
Input 1	Input 2	Target
0	0	0
0	1	1
1	0	1
1	1	0

XOR Plot



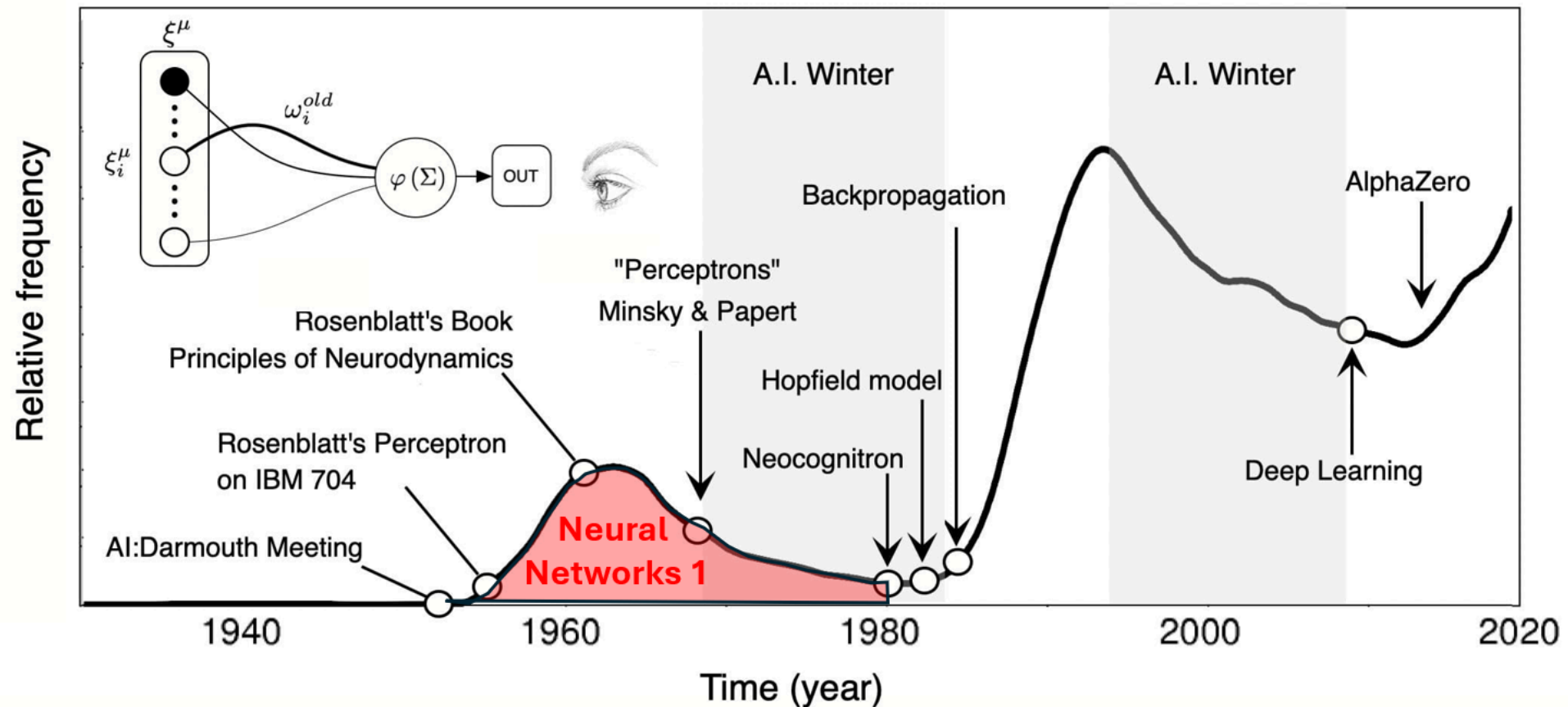
- could you draw a single straight line on this plot that puts both blue points on one side and both orange points on the other? Could a simple linear boundary (exactly what we learned to build today) resolve this?

The springs and winters of AI development



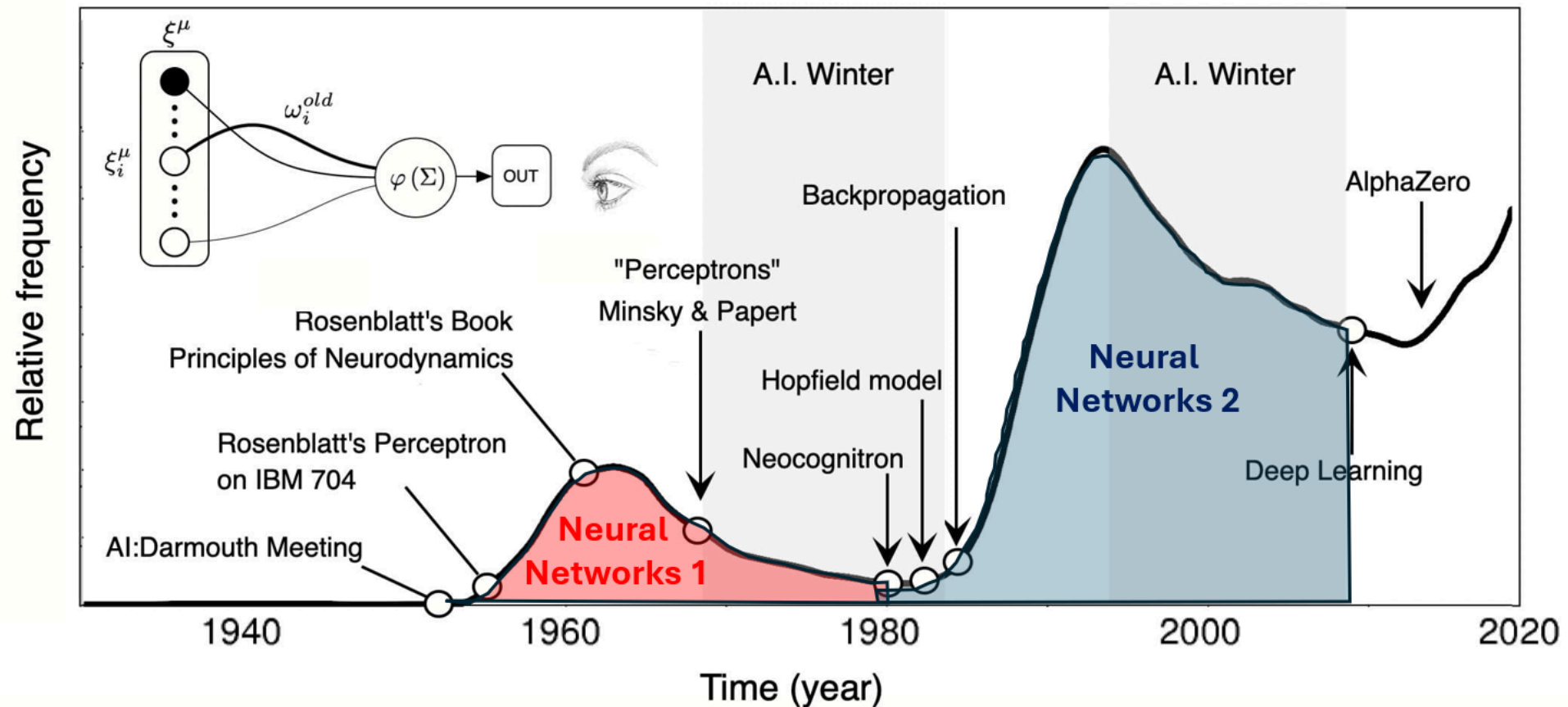
Minsky, M., & Papert, S. A. (1969). *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press.

The springs and winters of AI development



https://x.com/ricard_sole/status/1497306702181548039

The springs and winters of AI development



https://x.com/ricard_sole/status/1497306702181548039

Summary

- Connectionist models represent information as patterns of activation across units connected by weights; learning happens by changing weights, not by rewriting rules
- A single unit (a perceptron) is a weighed sum of inputs that goes through the sigmoid model with the same RMSD, same idea of parameters (now called weights and a bias) to be estimated from data
- The delta rule trains a unit by nudging its weights after each example: the error, times door 1 (that weight's own input), times door 2 (the sigmoid's steepness right now)
- Unlike `optim()`'s default Nelder-Mead (which never computes a derivative), the delta rule *is* genuine gradient descent. The door-2 steepness term is a real derivative (from calculus)
- A single unit can learn plenty of real categorization problems, but the tutorial will have you run it on the XOR problem (which is one of the causes of the first AI winter)
- Next week: adding a second layer of units fixes exactly this problem, and needs one more idea (backpropagation) to train
- Zoom out: today's 3-weight toy unit and the billion-parameter models behind current AI tools run on this same core mechanism. What changes from here is scale, not kind